



UNIVERSITY OF CAPE TOWN

CSC200SS: Computer Science

Author: George Goodey

October 8, 2026

Contents

1	Parallel and Concurrent Programming	3
2	Computer Architecture	3
2.1	Introduction	3
2.1.1	What is architecture?	3
2.1.2	Assembly programming	3
2.1.3	Components of a computer	3
2.1.4	Measuring performance	3
2.1.5	Performance fallacies	4
2.2	MIPS	4
2.2.1	Von Neumann architecture	4
2.2.2	Hierarchy of programming languages	5
2.2.3	Assembly	6
2.2.4	Instruction philosophies	6
2.2.5	MIPS	7
2.2.6	Assembler directives	8
2.2.7	Labels	8
2.2.8	Registers	8
2.2.9	Memory	9
2.2.10	Stack	11
2.2.11	Data transfer instructions	11
3	Theory of Computation	12
3.1	Regular Languages	12
3.1.1	Finite automata	12
3.1.2	Strings and languages	13
3.1.3	Regular expressions	13
3.1.4	Nondeterministic finite automata	14
3.1.5	Converting NFAs to DFAs	15
3.1.6	Closure properties for regular languages	15
3.1.7	Converting a regular expression to NFA	16
3.1.8	Generalized NFA	18
3.1.9	Converting a GNFA to a regular expression	19
3.1.10	Non-regular languages	19
3.1.11	Proving non-regularity	20
3.2	Context-free languages	22
3.2.1	Context-free grammars	22
3.2.2	Ambiguity	22
3.2.3	Pushdown automaton	23
3.2.4	Converting CFGs to PDAs	24
3.2.5	Equivalence of CFGs and PDAs	24
3.2.6	Chomsky normal form (CNF)	25
3.2.7	Proving languages are not context free	26
3.3	Turing Machines	28
3.3.1	Turing machines	28
3.3.2	Determinism	28

3.3.3	Instantaneous descriptions	29
3.3.4	Language accepted by a turing machine	29

1 Parallel and Concurrent Programming

me: where is the section, yo?

yo: gurt

gurt: idk

2 Computer Architecture

2.1 Introduction

2.1.1 What is architecture?

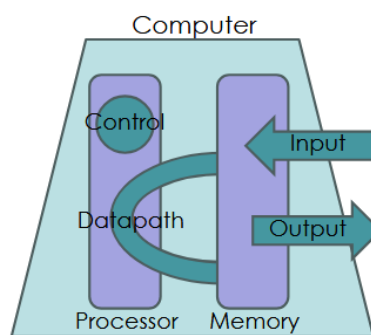
Computer architecture is a study of the overall structure and function of computer systems. In other words, how hardware components link and interact with software.

2.1.2 Assembly programming

A useful lens for understanding computer systems. It exposes the hardware but still (somewhat) device independent. High-level languages often compile first to assembly. It maps one-to-one to machine code instructions.

2.1.3 Components of a computer

- The **processor** gets instructions and data from memory.
- **Input** writes data from an external source to memory, and **output** reads data from memory and transmits to an external source.
- **Control** sends the signals that determine the operations.
- The **datapath** performs the data processing.



2.1.4 Measuring performance

The performance equation:

$$\text{Runtime} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}$$

where:

- IC is the instruction count, i.e. the total number of instructions in the program.
- CPI is the number of cycles per instruction, i.e. the average number of clock cycles required for an instruction.
- Clock Cycle Time = $\frac{1}{\text{Clock Rate}}$ is the time for one processor clock period.

This is an approximation of the actual measured runtime, but reasonably accurate.

2.1.5 Performance fallacies

Using a subset of the performance equation is problematic. The clock rate:

$$\text{Clock Rate} = \frac{\text{IC}}{\text{CPI} \times \text{Clock Cycle Time}}$$

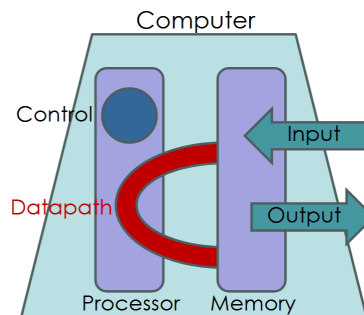
For example, the Intel Core i7-3770K has a clock rate of 3.9GHz. Different architectures may require different numbers of cycles for the same instructions. MIPS = Millions of Instructions Per Second:

$$\text{MIPS} = \frac{\text{IC}}{\text{CPI} \times \text{Clock Cycle Time}}$$

For example, the Intel Core i7-3770K can run 106924 MIPS. Different architectures has different instruction sets, so it cannot be used to directly compare program performance.

2.2 MIPS

2.2.1 Von Neumann architecture

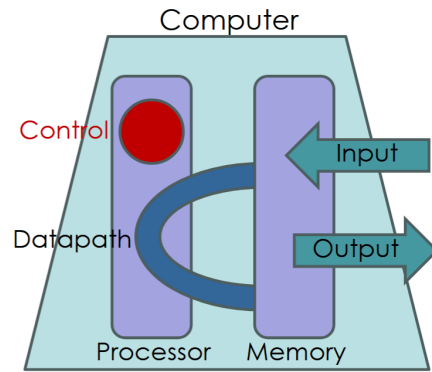


Datapath:

Instructions are data. Programs can manipulate other programs (e.g. compilers, debuggers, and OS).

Von Neumann bottleneck:

Data transfer takes much longer than data processing and the processor can sit idle. Memory hierarchy is an attempt at mitigation.



Control (Fetch-execute cycle):

- Fetch the next instruction from memory.
- Increment the program counter.
- Decode the instruction.
- Read operands.
- Execute operations.

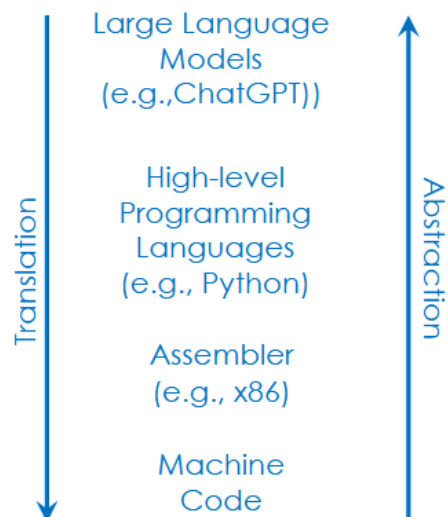
The program counter is a register containing the address of the instruction being executed.

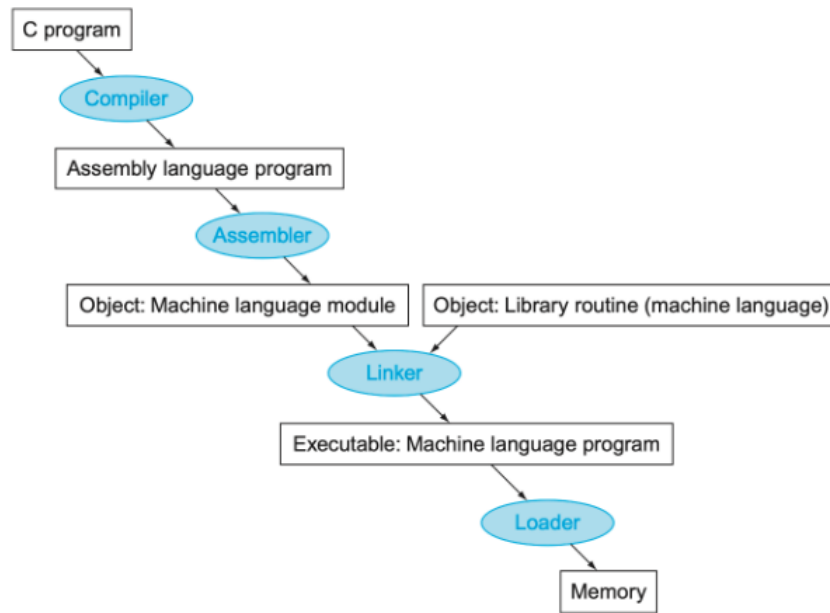
2.2.2 Hierarchy of programming languages

Greater abstraction allows for:

- More concise and natural expression of the solution.
- Greater machine independence.

But performance can still be an issue.





2.2.3 Assembly

Also known as "symbolic machine code." Any low-level programming language with a strong correlation to machine code instructions. There are different variants for each architecture:

- x86
- ARM
- MIPS
- PowerPC

There is usually a one-to-one mapping from assembly to machine code instructions.

MIPS:

```
add    $t0, $t1, $t2
mult   $t3, $t4
```

Machine code (base 16):

```
012a4020
016c0018
```

Machine code (base 2):

```
00000001001010100100000000100000
000000010110110000000000000011000
```

2.2.4 Instruction philosophies

Complex Instruction Set Computing (CISC):

- Larger, more complex instruction set.

- Easier to program, but leads to greater hardware complexity.
- Usually in older architecture, e.g. VAX, x86.

Reduced Instruction Set Computing (RISC):

- Simple instructions, executed rapidly.
- Usually in more modern architectures, e.g. ARM, MIPS.
- Generally, require more instructions for a given algorithm.

Important

There are **no guardrails** in assembly! it can be particularly brutal...

2.2.5 MIPS

MIPS stands for, Microprocessor without Interlocked Pipelined Stages.

- It is a family of RISC architectures.
- It is clean and scalable.
- Used in embedded devices and gaming consoles.
- It has a 32-bit and 64-bit version, but development stopped.

The general instruction format:

```
<operation>      <result> <operand1> <operand2> ...
```

Here is a simple MIPS addition program:

```
.data                                # Declare memory and constants
    num1:    .word    12            # Store 12 at memory location "num1"
    num2:    .word    13            # Store 13 at memory location "num2"
    num3:    .word    0             # Store 0 at memory location "num3"
.text                                # Code section
.globl main                          # Declare the label "main" as global

# Program that loads words from memory and stores words to memory

main:                                # Label "main" (starting point)
    lw       $t1, num1              # Load the value of num1 (12) in $t1
    lw       $t2, num2              # Load the value of num2 (13) in $t2
    add      $t3, $t1, $t2          # $t3 = $t1(12) + $t2(13) = 25
    sw       $t3, num3              # Store the value in $t3 (25) in num3

exit:                                # Label "exit"
    li       $v0, 10                # Load service 10 (terminate program)
    syscall                          # Perform system call with service 10.
```

2.2.6 Assembler directives

Top-level directives: Used to manage code structure

Directive	Description
.text	Indicates that the following items are stored in the user text segment, typically instructions.
.data	Indicates that the following data items are stored in the data segment.
.globl sym	Declare the symbol "sym" is global and can be referenced from other files.

Data directives:

On 32-bit architecture:

- byte = 8 bits
- halfword = 2 bytes
- word = 4 bytes

Directive	Description
.word w1,...,wn	Store n 4-byte quantities in successive memory locations.
.half h1,...,hn	Store n 2-byte quantities in successive memory locations.
.byte b1,...,bn	Store n 1-byte quantities in successive memory locations.
.ascii str	Store a non -null-terminated string in memory.
.asciiz str	Store a null-terminated string in memory.
.float f1,...,fn	Store n floating point single precision numbers in successive memory locations.
.double d1,...,dn	Store n floating point double precision numbers in successive memory locations.
.space n	Reserve n successive bytes of memory.

2.2.7 Labels

The notation of variables is a high-level programming concept not available in MIPS assembly. Instead, labels are used to identify addresses in data and code, e.g. "main" in code and "num1" in data. Importantly, RISC arithmetic and logic instructions do not operate on data memory directly but on registers instead. So, transfer from data memory to registers is a must!

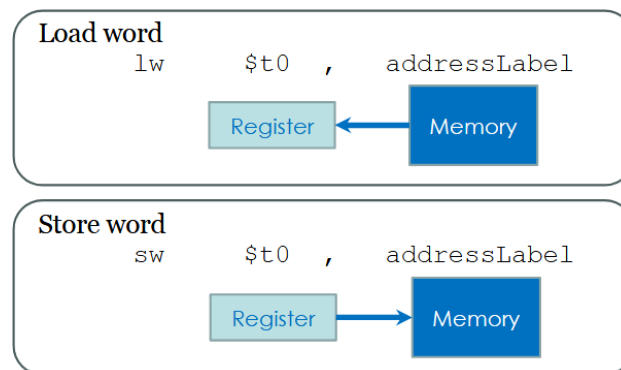
2.2.8 Registers

Processors contain a limited number of high-speed memory chips, called registers. MIPS32 has 32 general purpose and 32 floating point registers, each 32 bits wide. It is capable of addressing $2^{32} = 4,294,967,296 = 4\text{GB}$ of RAM. Unlike 64-bit which can address $\sim 18.1\text{EB}$ (exabytes).

Number	Name	Comments
\$0	\$zero, \$r0	Built into hardware to ALWAYS be zero.
\$1	\$at	Reserved for assembler.
\$2, \$3	\$v0, \$v1	First and second return values, respectively. Used for syscalls.
\$4, ..., \$7	\$a0, ..., \$a3	First four arguments to functions. Often used with syscalls.
\$8, ..., \$15	\$t0, ..., \$t7	Temporary registers.
\$16, ..., \$23	\$s0, ..., \$s7	Saved registers (must be restored after function calls).
\$24, \$25	\$t8, \$t9	More temporary registers.
\$26, \$27	\$k0, \$k1	Reserved for kernel (operating system).
\$28	\$gp	Global pointer.
\$29	\$sp	Stack pointer.
\$30	\$fp	Frame pointer.
\$31	\$ra	Return address.
	\$f0, ..., \$f32	Single precision floating point registers.

2.2.9 Memory

Operations can only be performed on register, thus we need to move values from memory to registers before use.



Memory structure:

Logically, a 1D array of bytes. Labels map to addresses in memory. Less significant bits placed at lower addresses (little endian).

For example, \$t0 holds number 0x65676361. \$s0 holds address 12. Then,

```
sw    $t0, $s0
```

result in the following:

Address	Value
19	
18	
17	
16	
15	65
14	67
13	63
12	61
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

Each new label in *.data* is aligned on type-specific boundary. For example, word addresses are multiples of 4. But, comma separated values are packed as tightly as possible.

Memory access with offsets:

We need offset instructions to address elements in a sequence of numbers. For example:

```
arr:    .word 11, 12, 13
```

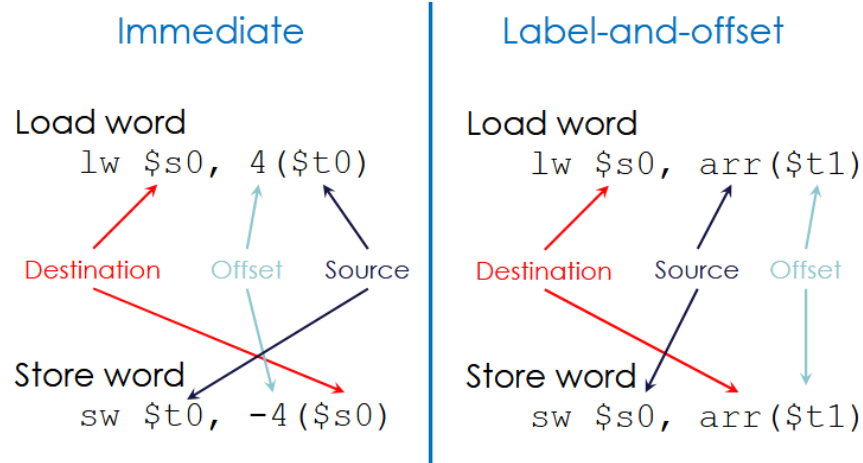
Here *arr* is the address of the first element. Much like indexing an array. We can use byte offsets to address elements.

Immediate form:

```
lw      $s0, 4($t0) # $t0 holds address of arr
```

Label and offset form:

```
lw      $s0, arr($t1) # if $t1 holds 4
```



2.2.10 Stack

We often need to free up registers but retain their values. We can use stack memory:

- Push registers onto the stack.
- Pop values from the stack back into registers.
- Needs to be managed by hand using the register $\$sp$ pointing to the end of the stack.
- Stack grows from top downward.

Stack Push:

Push 2 words. $\$sp$ is initially at 12

Code:

```
1. addi $sp, $sp, -8
2. sw   $t0, 4($sp)
3. sw   $t1, 0($sp)
```

Logic:

1. Decrement $\$sp$ by 8 bytes
2. Store $\$t0$ at address $\$sp+4$
3. Store $\$t1$ at address $\$sp$

Initial $\$sp \rightarrow$

Final $\$sp \rightarrow$

Address	Value
15	
14	
13	
12	
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

Stack Pop:

Pop 2 words. $\$sp$ is initially at 4

Code:

```
1. lw   $t3, 0($sp)
   lw   $t4, 4($sp)
   addi $sp, $sp, 8
```

Logic:

1. Load value from address $\$sp$ into $\$t3$
2. Load value from address $\$sp+4$ into $\$t4$
3. Increment $\$sp$ by 8 bytes

Final $\$sp \rightarrow$

Initial $\$sp \rightarrow$

Address	Value
15	
14	
13	
12	
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

2.2.11 Data transfer instructions

Instruction	Example	Comments
-------------	---------	----------

3 Theory of Computation

3.1 Regular Languages

3.1.1 Finite automata

Definition: Finite automata

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

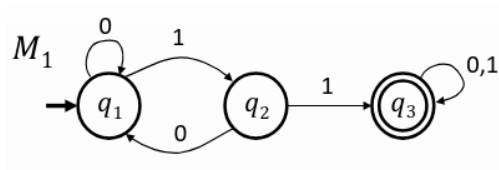
- Q is a finite set of all possible states.
- Σ is a finite set of all alphabet symbols.
- δ is a transition function $\delta : Q \times \Sigma \rightarrow Q$. In other words,

$$\delta(q, a) = r \text{ means } q \xrightarrow{a} r$$

- q_0 is the start state.
- F is the set of accept states.

Example:

Let's consider the following finite automaton.



Here, $M_1 = (Q, \Sigma, \delta, q_1, F)$ is a finite automaton. For sake of example, here are the exact details of M_1 .

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- $F = \{q_3\}$

It accepts a finite string as input. The output is either *ACCEPT* or *REJECT*. The computation goes as follows:

1. Begin at start state.
2. Read input symbols.
3. Follow corresponding transitions.
4. *ACCEPT* if we reach the accept state, otherwise *REJECT*.

For example: $01101 \rightarrow \text{ACCEPT}$, while $00101 \rightarrow \text{REJECT}$. Here, M_1 accepts all string in A where $A = \{w | w \text{ contains substring } 11\}$. We say that A is the language of $M_1 \iff M_1$ recognizes $A \iff A = L(M_1)$.

3.1.2 Strings and languages

A **string** is a finite sequence of symbols $w_1w_2\cdots w_n \in \Sigma$. A **language** is a set of string (finite or infinite). The **empty string** ε is the string of length 0. The **empty language** $\emptyset$ is the set with no strings.

Definition: Accepting strings

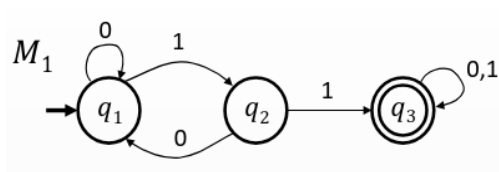
A finite automaton M **accepts string** $w = w_1w_2\cdots w_n$ for each $w_i \in \Sigma$ if there exists a sequence of states $r_0, r_1, \dots, r_n \in Q$ where:

- $r_0 = q_0$
- $r_i = \delta(r_{i-1}, w_i)$ for $1 \leq i \leq n$
- $r_n \in F$

Definition: Regular language

We say a language is **regular** if some finite automaton recognizes it.

Example 1:



$L = (M_1) = \{w | w \text{ contains substring } 11\} = A$. Therefore, A is regular.

Example 2:

Let $B = \{w | w \text{ has an even number of 1s}\}$. In this case, B is regular. (not shown)

Example 3:

Let $C = \{w | w \text{ has equal number of 0s and 1s}\}$. In this case, C is not regular. (provable)

3.1.3 Regular expressions

Definition: Regular operations

Let A, B be languages.

- **Union:** $A \cup B = \{w | w \in A \text{ or } w \in B\}$.
- **Concatenation:** $A \circ B = \{xy | x \in A \text{ and } y \in B\} = AB$.
- **Star:** $A^* = \{x_1 \dots x_k | x_i \in A \text{ for } k \geq 0\}$. (Note: $\varepsilon \in A^*$)

Regular expressions are built from $\Sigma, \emptyset, \varepsilon$ by using regular operations $\cup, \circ, *$.

Example 1:

- $(0 \cup 1)^* = \Sigma^*$ gives all strings over Σ .
- Σ^*1 gives all strings that end with 1.
- $\Sigma^*11\Sigma^*$ gives all strings that contain 11 = $L(M_1)$.

Example 2:

Let $A = \{\text{good, bad}\}$ and $B = \{\text{boy, girl}\}$.

- $A \cup B = \{\text{good, bad, boy, girl}\}$.
- $A \circ B = AB = \{\text{goodboy, goodgirl, badboy, badgirl}\}$.
- $A^* = \{\varepsilon, \text{good, bad, goodgood, goodbad, badgood, badbad, goodgoodgood, ...}\}$

3.1.4 Nondeterministic finite automata**Definition: Nondeterministic finite automata**

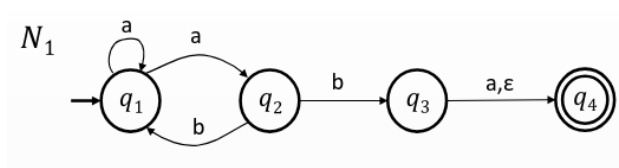
A **nondeterministic finite automata (NFA)** N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where everything is the exact same as a DFA except δ . Here, $\delta : Q \times \Sigma_\varepsilon \rightarrow \mathcal{P}(Q) = \{R \mid R \subseteq Q\}$.

New features of nondeterminism:

- Multiple paths possible (0, 1, or many at each step)
- ε -transition is a "free" move without reading input.
- Accept input if **some** path leads to the accept state.

Nondeterminism doesn't correspond to a physical machine we can build. However, it is useful mathematically. Here are some ways to think about it:

- **Computationally:** Fork new parallel thread and accept if any thread leads to an accept state.
- **Mathematically:** Tree with branches. Accept if any branch leads to an accept state.
- **Magical:** Guess at each nondeterministic step which way to go. The machine always makes the right guess that leads to the accept state, if possible.

Example:

Here, we have $\delta(q_1, a) = \{q_1, q_2\}$, but $\delta(q_1, b) = \emptyset$.

3.1.5 Converting NFAs to DFAs

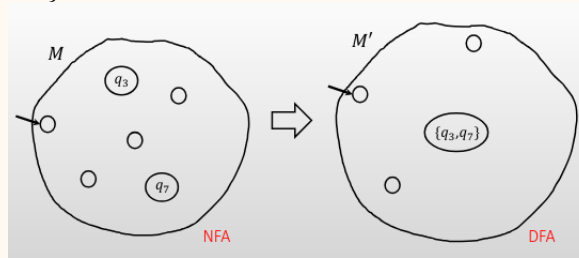
Theorem

If an NFA recognizes A , then A is regular.

Proof:

Let NFA $N = (Q, \Sigma, \delta, q_0, F)$ recognize A . We need to construct a DFA $M = (Q', \Sigma, \delta', q'_0, F')$, ignoring the ε -transitions. Here M is defined as:

- $Q' = \mathcal{P}(Q)$.
- $\delta'(R \in Q', a) = \{q \in Q \mid \delta(r, a) = q \text{ for some } r \in R\}$.
- $q'_0 = \{q_0\}$.
- $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$.



3.1.6 Closure properties for regular languages

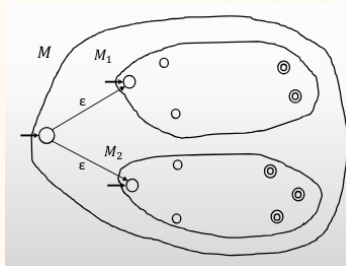
Theorem: Closure under $\cup$.

If A_1 and A_2 are regular languages, then so is $A_1 \cup A_2$.

Proof:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 . We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where

- $Q = Q_1 \times Q_2 = \{(q_1, q_2) \mid q_1 \in Q_1 \wedge q_2 \in Q_2\}$.
- $q_0 = (q_1, q_2)$.
- $\delta((q, r), a) = (\delta_1(q, a), \delta_2(r, a))$.
- $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$.

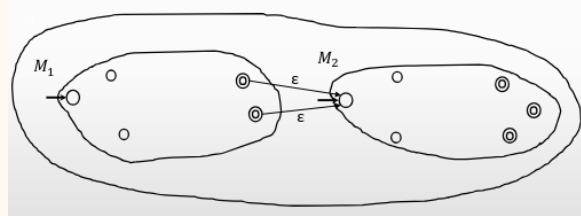


Theorem: Closure under $\circ$.

If A_1 and A_2 are regular languages, then so is A_1A_2 .

Proof:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 and similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 . We need to construct $M = (Q, \Sigma, \delta, q_0, F)$ where



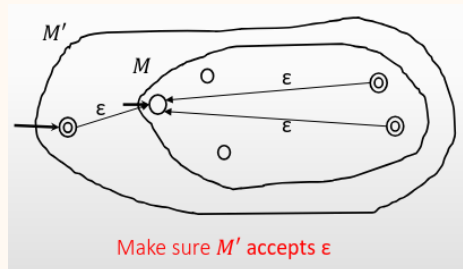
M should accept input w if $w = xy$ where M_1 accepts x and M_2 accepts y . The nondeterministic M has the option to jump to M_2 when M_1 accepts.

Theorem: Closure under $*$

If A is a regular language, then so is A^* .

Proof:

Given a DFA M recognizing A , construct an NFA M' recognizing A^* .



M' should accept input w if $w = x_1x_2 \dots x_k$ where $k \geq 0$ and M accepts each x_i .

3.1.7 Converting a regular expression to NFA

Theorem

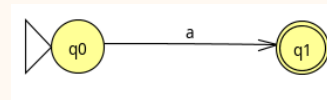
If R is a regular expression and $A = L(R)$, then A is regular.

Proof:

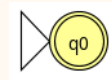
Convert R to the equivalent NFA M .

If R is atomic, an equivalent M is:

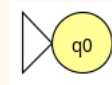
- $R = a$ for $a \in \Sigma$



- $R = \varepsilon$



- $R = \emptyset$



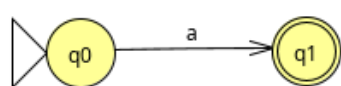
If R is composite:

- $R = R_1 \cup R_2$
- $R = R_1 \circ R_2$
- $R = R_1^*$

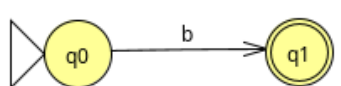
Example:

Convert $(a \cup ab)^*$ to an equivalent NFA.

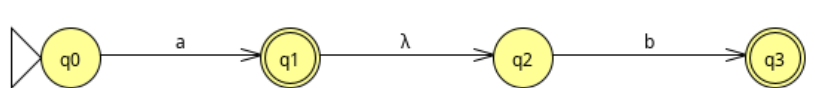
- a :



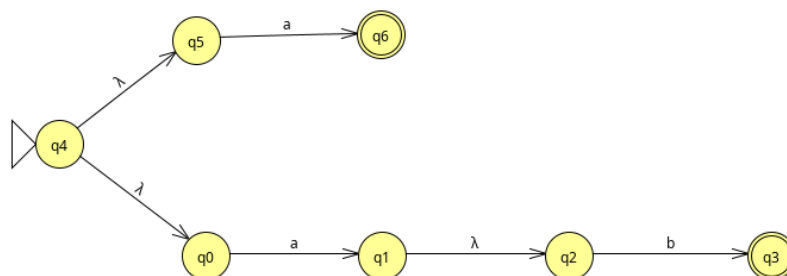
- b :



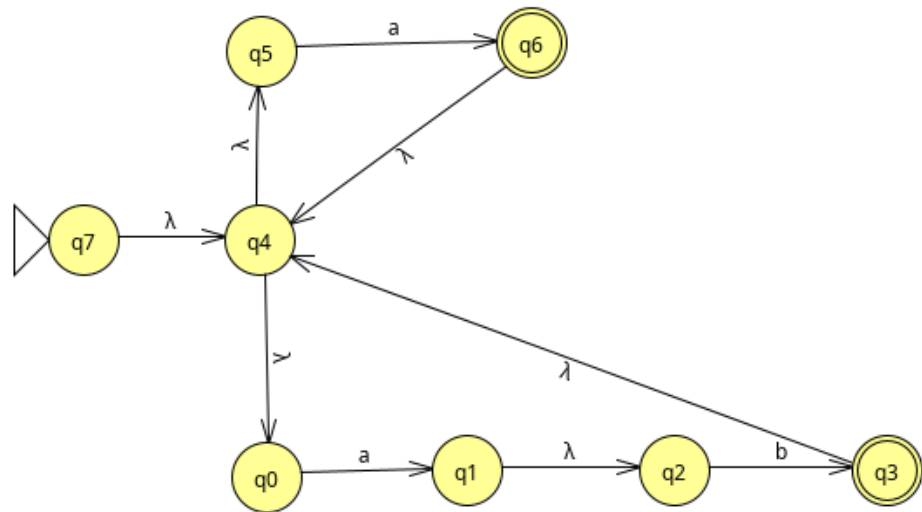
- ab :



- $a \cup ab$:



- $(a \cup ab)^*$:



Reminder

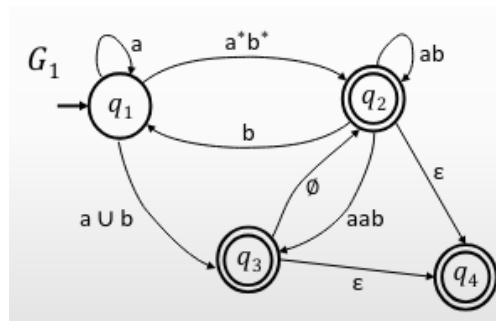
A λ transition is equivalent to an ε transition.

3.1.8 Generalized NFA

Definition: Generalized NFA

A **generalized nondeterministic finite automaton (GNFA)** is similar to an NFA, but allows regular expressions as transition labels.

Example:



For convenience, we will assume:

- One accept state, separate from the start state.
- One arrow from each state to each state, except
 - only exiting the start state
 - only entering the accept state

We can easily modify a GNFA to have this special form.

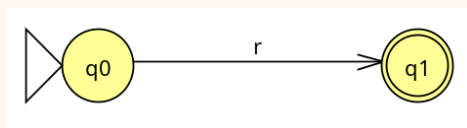
3.1.9 Converting a GNFA to a regular expression

Lemma

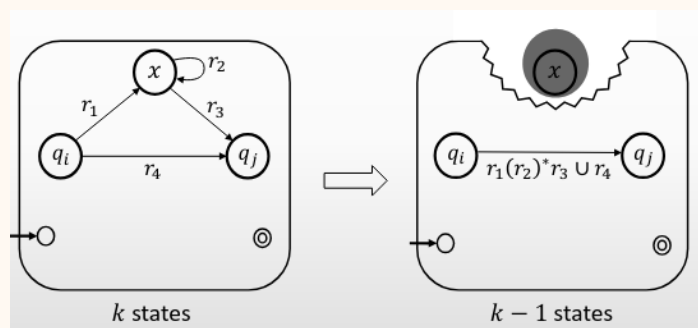
Every GNFA G has an equivalent regular expressions R .

Proof:

We will make use of an inductive argument on the number of states k of G . For the base step ($k = 2$):



Assume the Lemma holds true for $k - 1$ states. Now we want to prove it holds for k states. The central idea is to convert a k -state GNFA to an equivalent $(k - 1)$ -state GNFA.



1. Pick any state x except the start and accept states.
2. Remove x .
3. Repair the damage by recovering all paths that went through x .
4. Make the indicated change for each pair of states q_i, q_j .

Thus DFAs and regular expressions are equivalent. ■

3.1.10 Non-regular languages

How do we show a language is not regular?

- Remember, to show a language is *regular*, we give an equivalent DFA.
- To show a language is *not regular*, we must give a proof.
- It is not enough to say you couldn't find a DFA for it, therefore it isn't regular!

Example 1:

Let $B = \{w | w \text{ has equal numbers of 0s and 1s}\}$. One's intuition says that B is not regular since DFAs cannot count unboundedly.

Example 2:

Let $C = \{w \mid w \text{ has equal numbers of } 01 \text{ and } 10 \text{ substrings}\}$. As examples, $0101 \notin C$ and $0110 \in C$. Again, one's intuition says that C is not regular since DFA cannot count unboundedly. However, C is regular!

3.1.11 Proving non-regularity**Pumping Lemma**

For every regular language A , there exists a number p , called the "pumping length", such that if $s \in A$ and $|s| \geq p$ then $s = xyz$ where

$$1. \ xy^iz \in A \text{ for all } i \geq 0 \quad (y^i = \underbrace{yy \cdots y}_i).$$

$$2. \ y \neq \varepsilon.$$

$$3. \ |xy| \leq p.$$

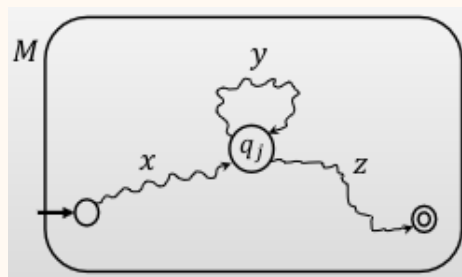
Informally, if A is regular, then every long string in A can be "pumped" and the result stays in A .

Proof:

Let DFA M recognize A . Let p be the number of states in M . Pick $s \in A$ where $|s| \geq p$.

$$s = \begin{array}{c} x \quad y \quad z \\ \hline q_j \quad q_j \end{array}$$

M will repeat a state q_j when reading s because s is so long. The path that M follows when reading s .

**Example 1:**

Let $D = \{0^k 1^k \mid k \geq 0\}$. Show that D is not regular.

Assume, for the sake of contradiction, that D is regular.

Since D is regular, the pumping lemma must hold. Let $s = 0^p 1^p \in D$.

The pumping lemma says that we can divide $s = xyz$.

But $xyyz$ has excess 0s, and thus $xyyz \notin D$ contradicting the pumping lemma.

Therefore, our assumption is false and thus D is not regular.

Example 2:

Let $F = \{ww \mid w \in \Sigma^*\}$. Say $\Sigma^* = \{0, 1\}$. Show that F is not regular.

Assume, for the sake of contradiction, that F is regular.

Since F is regular, the pumping lemma must hold.

We need to choose $s \in F$. Let $s = 0^p 10^p 1 \in F$.

We can show that $s = xyz$ cannot be pumped satisfying the 3 conditions.

Thus, $xyyz \notin F$, which is a contradiction. Therefore, F is not regular.

Example 3:

Let $B = \{w \mid w \text{ has equal numbers of 0s and 1s}\}$. Show that B is not regular.

Assume, for the sake of contradiction, that B is regular.

We know 0^*1^* is regular, and so $B \cap 0^*1^*$ is regular (closure under intersection).

But $D = B \cap 0^*1^*$ and we already showed D is not regular (from example 1).

Therefore, B is not regular.

Example 4:

Let $A = \{1^{n^2} \mid n \geq 0\}$. (*A contains all strings of 1s whose length is a perfect square*). Show that A is not regular.

Assume, for the sake of contradiction, that A is regular.

Let p be the pumping length given by the pumping lemma. Take $s = 1^{p^2} \in A$.

The pumping lemma guarantees $s = xyz$ with $|s| = |xy^i z| \in A$ for some $i \geq 0$.

Consider the two strings xyz and xy^2z . The length of these strings differ by y .

By condition 3, $|xy| \leq p$ and thus $|y| \leq p$. Then, we have $|xyz| = p^2$ and so $|xy^2z| \leq p^2 + p$.

But $p^2 + p < p^2 + 2p + 1 = (p+1)^2$.

Moreover, condition 2 implies that y is not the empty string and so $|xy^2z| > p^2$.

Therefore, the length of xy^2z lies strictly between the consecutive squares p^2 and $(p+1)^2$.

Hence this length cannot be a perfect square itself, which is a contradiction.

Thus, $xy^2z \notin A$ and so we conclude that A is not regular.

Example 5:

Let $E = \{0^i 1^j \mid i > j\}$. Show that E is not regular.

Assume, for the sake of contradiction, that E is regular.

Let p be the pumping length given by the pumping lemma. Take $s = 0^{p+1} 1^p \in E$.

By condition 1, $xy^i z \in E$ even when $i = 0$. Consider the string $xy^0 z = xz$.

Removing string y decreases the number of 0s in s .

But, recall that s has just one more 0 than 1.

Therefore, xz cannot have more 0s than 1s and so $s = 0^{p+1} 1^p \notin E$.

Hence a contradiction. Therefore, E is not regular.

3.2 Context-free languages

3.2.1 Context-free grammars

Definition: Context-free grammar

A **Context Free Grammar (CFG)** G is a 4-tuple (V, Σ, R, S) where

- V is the finite set of variables.
- Σ is the finite set of terminal symbols.
- R is the finite set of rules (rule form: $V \rightarrow (V \cup \Sigma)^*$).
- S the starting variable.

For $u, v \in (V \cup \Sigma)^*$ write

1. $u \Rightarrow v$ if you can go from u to v with one substitution step in G .
2. $u \Rightarrow^* v$ if you can go from u to v with some number of substitution steps in G .
 $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_k \Rightarrow v$ is called a *derivation* of v from u . Furthermore, if $u = S$ then it is a *derivation* of v .

$$L(G) = \{w | w \in \Sigma^* \text{ and } S \Rightarrow^* w\}.$$

Definition: Context-free language

A is a **Context Free Language (CFL)** is $A = L(G)$ for some CFG G .

3.2.2 Ambiguity

Take the following two CFGs:

G_1

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T \times F \mid F$$

$$F \rightarrow (E) \mid a$$

G_2

$$E \rightarrow E + E \mid E \times E \mid (E) \mid a$$

Both G_1 and G_2 recognize the same language, i.e. $L(G_1) = L(G_2)$. However, G_1 is an unambiguous CFG and G_2 is ambiguous.

3.2.3 Pushdown automaton

Definition: Pushdown automaton

A **Pushdown Automaton (PDA)** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$:

- Σ is the input alphabet.
- Γ is the stack alphabet.
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$
 $\delta(q, a, c) = \{(r_1, d), (r_2, e)\}$

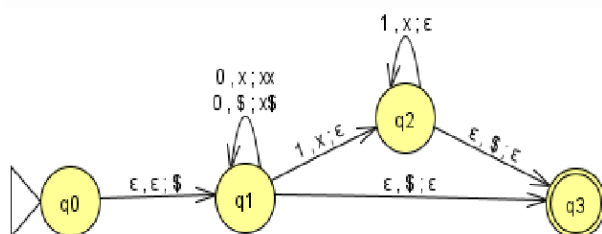
We accept if some thread is in the accept state at the end of the input string.

A PDA operates like an NFA except can write/add (push) or read/remove (pop) symbols from the top of the stack. The input appears on a tape.

Example 1:

PDA for $D = \{0^k 1^k : k \geq 0\}$

1. Read 0s from input, push 0 onto the stack until a 1 is read.
2. Read 1s from input, while popping 0s from the stack.
3. Enter the accept state if the stack is empty.



How to read transitions for PDAs

Example 1: Take the transition $\epsilon, \epsilon; \$$.

- The first ϵ is the input read from the tape. Since we're reading ϵ , this is equivalent to a blank on the tape.
- The second ϵ is the value on the top of the stack. Since we're reading ϵ , we assume the stack is empty.
- The last $\$$ is part of the stack alphabet. This signifies the bottom of the stack. We want to "push" this value onto the stack.

Example 2: Take the transition $\epsilon, \$; \epsilon$.

- The first ϵ is the input read from the tape. Since we're reading ϵ , this is equivalent to a blank on the tape.
- The $\$$ is the value we expect on the top of the stack.

- The second ε is the value we push onto the stack. We're effectively replacing the bottom-of-stack character with a blank, clearing the stack.

Example 2:

PDA for $B = \{ww^R : w \in \{0,1\}^*\}$ with sample input: 011110.

1. Read and push input symbols onto the stack. Non-deterministically either repeat or go to (2).
2. Read input symbols and pop stack symbols, and compare. If they are ever not equal, then the thread rejects.
3. Enter the accept state if the stack is empty.

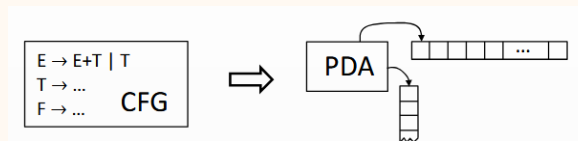
3.2.4 Converting CFGs to PDAs

Theorem

If A is a CFG, then some PDA recognizes A .

Proof:

Convert A 's CFG to a PDA.



1. Push the start symbol onto the stack.
2. If the top of the stack is:
 - *Variable*: replace with right hand side of rule (non-deterministic choice).
 - *Terminal*: pop it and match with next input symbol.
3. If the stack is empty, enter the accept state.

3.2.5 Equivalence of CFGs and PDAs

Theorem

A is a CFL if and only if some PDA recognizes A . (*Proof not examinable*)

Corollaries

1. Every regular language is a CFL.
2. If A is a CFL and B is regular, then $A \cap B$ is a CFL.

Note 1: If A and B are CFLs then $A \cap B$ may not be a CFL. Therefore, the class of CFLs is not closed under $\cap$.

Note 2: The class of CFL is closed under $\cup, \circ, *$.

3.2.6 Chomsky normal form (CNF)

Chomsky normal form is a simplification of grammars:

- If the start symbol is recursive, remove it.
- Removing ε -productions.
- Removing unit productions.
- Removing "useless" productions.

Converting to CNF

Given a grammar $G = \{V, T, S, P\}$, we can derive $G' = \{V, T, S', P'\}$ as follows:

Removing recursive start symbol:

Introduce a new start symbol S' . Let P' be the production rule set P of G with the following rule added to it: $S' \rightarrow S$.

Remove ε -productions:

If there is a production of the form $A \rightarrow \varepsilon$ then A is a nullable variable. If $B \rightarrow A_1 A_2 \cdots A_n$ and $A_1 A_2 \cdots A_n$ are nullable variables, then B is also a nullable variable. A given grammar G can be converted into a grammar G' that does not contain nullable variables.

1. Remove each productions rule of the form $A \rightarrow \varepsilon$.
2. Add each rule of the form $A \rightarrow s_1 s_2 \cdots s_n$ with $s_i \in V \cup T$ such that each s_i is not a nullable variable to P' .
3. For each production rule $A \rightarrow s_1 N_1 s_2 N_2 \cdots s_n N_n$, where each s_i is not a nullable variable and $N_1, N_2, \dots, N_n$ are nullable variables, add a copy of the rule with one or more N_i 's replaced by ε to P' , e.g. $A \rightarrow s_1 s_2 N_2 \cdots s_n N_n$. Note that if the s_i 's are also nullable, then we do not add $A \rightarrow \varepsilon$ unless A is the start symbol.

Remove unit productions:

- Draw a dependency graph with edge (A, B) for each unit production of the form $A \rightarrow B$.
- From the graph, identify all $A \rightarrow B$.
- For all A and B identified, add a production of the form $A \rightarrow s_1 \mid s_2 \mid \cdots \mid s_n$ to P' where $B \rightarrow s_1 \mid s_2 \mid \cdots \mid s_n$. Note that each $s_i \in V \cup T$ and none of the s_i form unit productions.

Remove useless productions:

There are essentially two types of useless productions:

- Productions containing variables that do not terminate.
- Productions containing variables that are not reachable.

Identify these productions are remove them from P' .

Correct production rule format:

A context-free grammar G is in CNF, if the production rules have the following form:

- $A \rightarrow AB$ where A and B are variables, or
- $A \rightarrow a$ where a is a single terminal.

Example:

Take the following grammar:

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

Step 1: Remove recursive start symbol

$$S' \rightarrow S$$

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

Step 2: Remove ε -productions

$$S' \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid AS \mid SA \mid S \mid a$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

Step 3: Remove unit productions

$$S' \rightarrow ASA \mid aB \mid AS \mid SA \mid a$$

$$S \rightarrow ASA \mid aB \mid AS \mid SA \mid a$$

$$A \rightarrow b \mid ASA \mid aB \mid AS \mid SA \mid a$$

$$B \rightarrow b$$

Step 4: Ensure correct production rule format

$$S' \rightarrow AA1 \mid UB \mid AS \mid SA \mid a$$

$$S \rightarrow AA1 \mid UB \mid AS \mid SA \mid a$$

$$A \rightarrow b \mid AA1 \mid UB \mid AS \mid SA \mid a$$

$$A1 \rightarrow SA$$

$$U \rightarrow a$$

$$B \rightarrow b$$

3.2.7 Proving languages are not context free

Pumping Lemma for CFLs

For every CFL A , there is a p such that if $s \in A$ and $|s| \geq p$ then $s = uvxyz$ where

1. $uv^i xy^i z \in A$ for all $i \geq 0$.
2. $vy \neq \varepsilon$.
3. $|vxy| \leq p$.

Informally, all long string in A are pumpable and stay in A .

Example 1:

Let $B = \{0^k 1^k 2^k \mid k \geq 0\}$. Show that B is not a CFL.

Assume, for the sake of contradiction, that B is a CFL.

The CFL pumping lemma gives p as above. Let $s = 0^p 1^p 2^p \in B$.

The pumping lemma says that we can divide $s = uvxyz$ satisfying the 3 conditions.

Condition 3 ($|vxy| \leq p$) implies that vxy cannot contain both 0s and 2s.

So $uv^2 xy^2 z \notin B$, violating condition 1, which is a contradiction.

Therefore, our assumption is false. We conclude that B is not a CFL.

Example 2:

Let $F = \{ww \mid w \in \Sigma^*\}$ where $\Sigma = \{0, 1\}$. Show that F is not a CFL.

Assume, for the sake of contradiction, that F is a CFL.

The CFL pumping lemma gives p as above. Take $s = 0^p 1^p 0^p 1^p \in F$.

Condition 3 implies that vxy does not overlap two runs of 0s or two runs of 1s.

Therefore, in $uv^2 xy^2 z$, two runs of 0s or two runs of 1s have unequal length.

So $uv^2 xy^2 z \notin F$ violating Condition 1, which is a contradiction.

Thus, F is not a CFL.

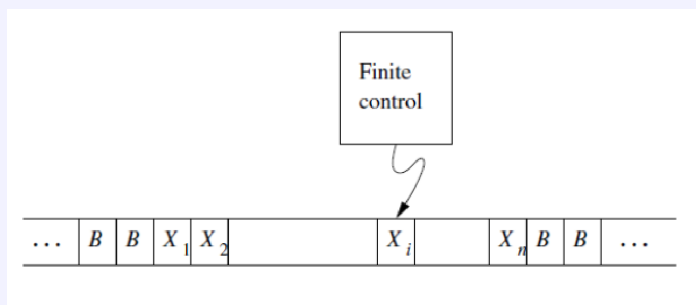
3.3 Turing Machines

3.3.1 Turing machines

Definition: Turing machines

A **turing machine** M is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ where:

- Q is the set of states.
- Σ is the input alphabet (*finite*).
- Γ is the tape alphabet (*finite*).
- δ is the transition function.
- $q_0 \in Q$ is the initial/start state.
- $F \subseteq Q$ is the final state(s).
- $\square \in \Gamma$ is the blank symbol.



There some conditions:

- $\Sigma \subseteq \Gamma$, since the input is written on the tape initially.
- $\square \in \Gamma \setminus \Sigma$, since the rest of the tape is blank.

Initially, we're in state q_0 . The tape contains input w surrounded by $\square$. The tape head is in the left-most cell of the input also denoted with D . The transitions work as follows:

- $\delta : Q \times \Gamma \Rightarrow Q \times \Gamma \times \{L, R\}$.
- $\delta(q, x) = (p, y, d)$ means that if M is in state q and the tape head is over symbol x , then M :
 1. Changes state to p .
 2. Replaces x by y on the tape.
 3. Moves the tape head by one cell in a direction d (*Either left or right*).

3.3.2 Determinism

A turing machine is **deterministic** if, for each $\delta(q, x)$ we have at most one move ($\delta(q, x)$ could also be undefined).

Acceptance:

w is accepted by a turing machine M if M , when started with w on the tape, eventually enters a final state. We can assume that all final states are halting, i.e. no transition is defined for them.

Rejection:

M halts in non-final states. In other words, in states for which no transition is defined, or it never halts (*Infinite loop*).

3.3.3 Instantaneous descriptions

This is the configuration of a turing machine. It describes the current situation of the turing machine (and the tape).

$$ID = \alpha q \beta \text{ with } q \in Q, \alpha, \beta \in \Gamma^*$$

Meaning:

- The non-blank portion of the tape contains α, β .
- The head is on the left-most symbol of β .
- The machine is in state q .

We will use $\vdash$ and $\vdash^*$ to denote the changes.

3.3.4 Language accepted by a turing machine

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a turing machine. Then, the language accepted by M is

$$L(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \alpha p \beta \text{ with } p \in F \text{ and } \alpha, \beta \in \Gamma^*\}$$

The set of all languages we can accept using a turing machine is often called the recursively enumerable languages. Conversely, a turing machine can accept a language by halting:

$$H(M) = \{w \in \Sigma^* \mid q_0 w \vdash^* \alpha p \beta \text{ where there is no possible move from } \alpha p \beta\}$$